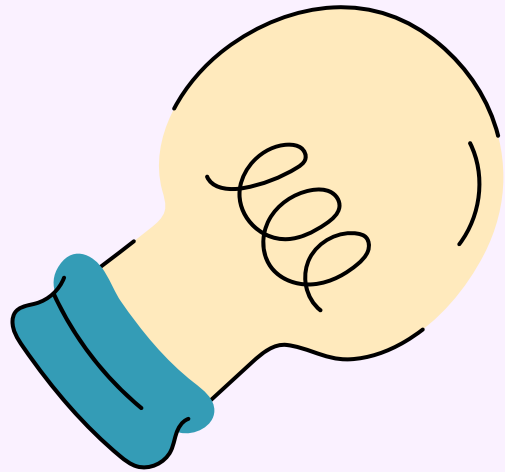
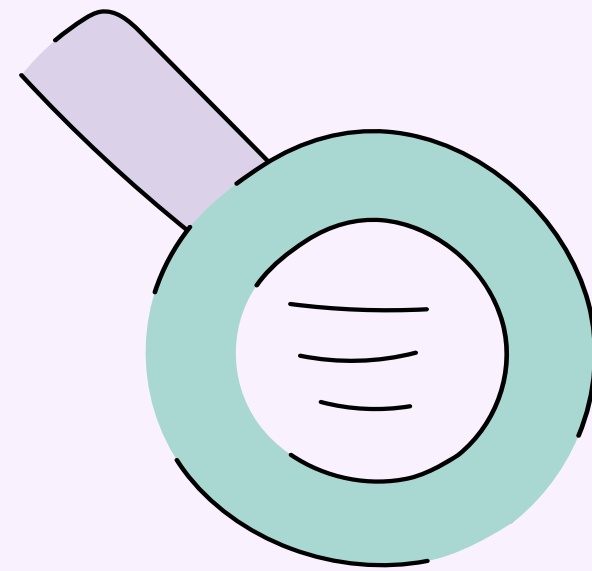


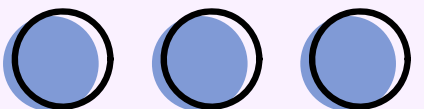
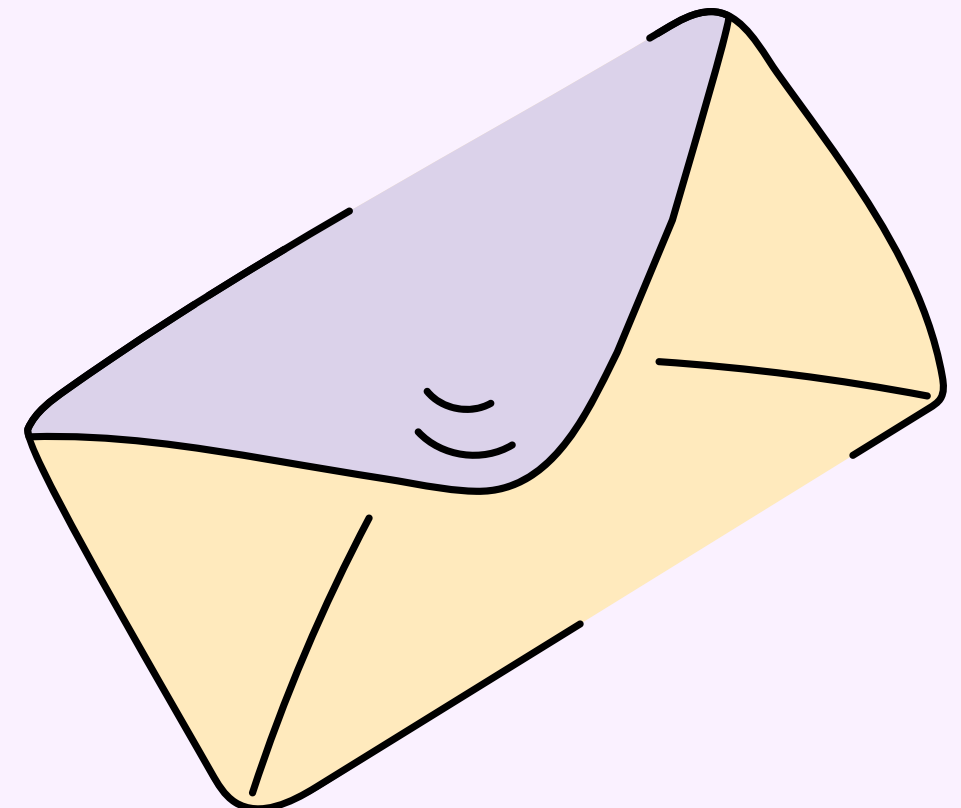
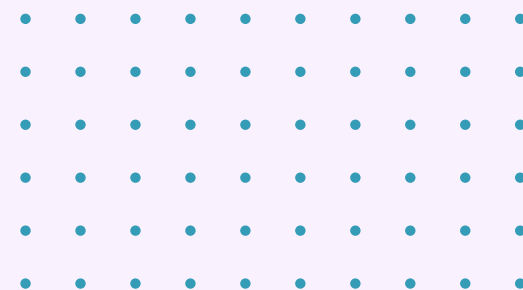
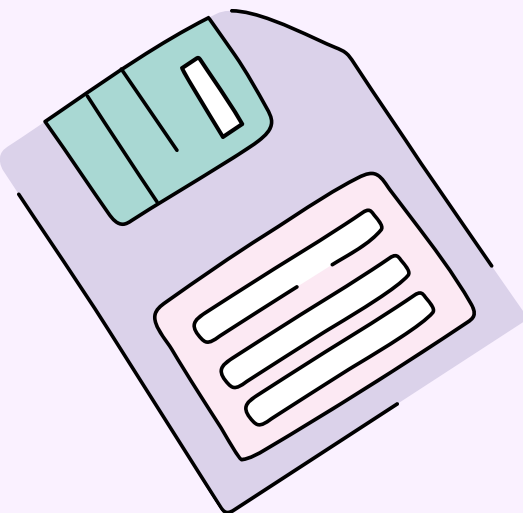


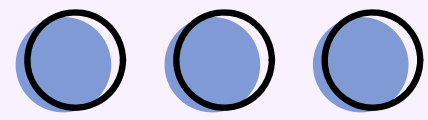
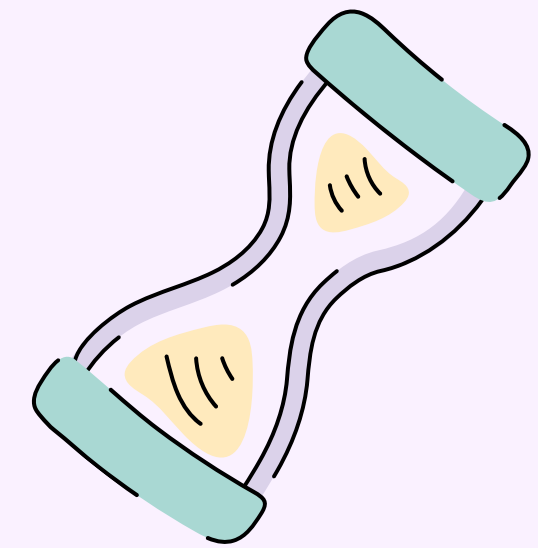
Exception Handling



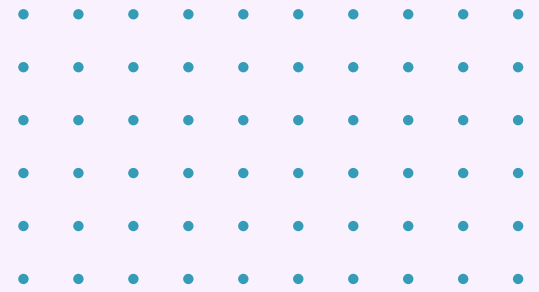
Presented by Sara Abbasghorbani

Advanced Programming - Spring 2026 - Dr Vahidi Asl





What will we learn in this session?



1) Bugs | Errors | Exceptions

2) Exception's Hierarchy

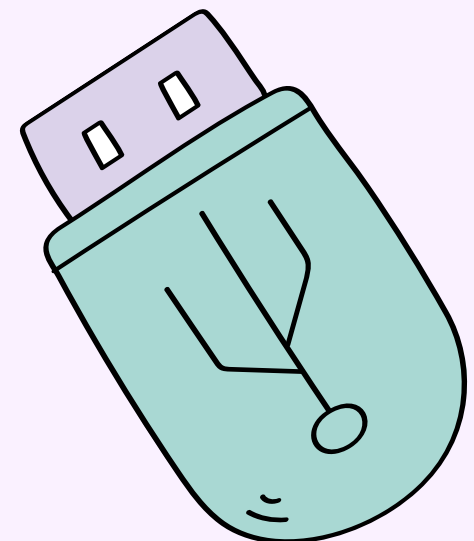
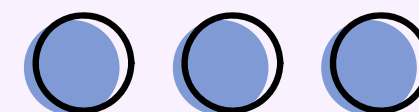
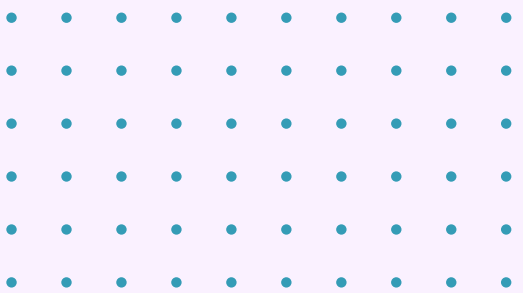
3) Checked Exceptions &
Unchecked Exceptions

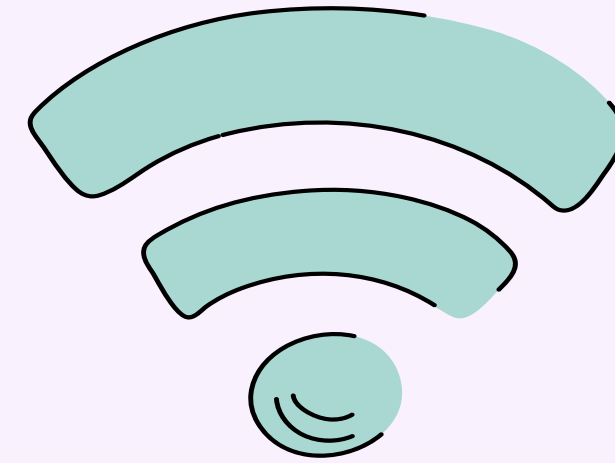
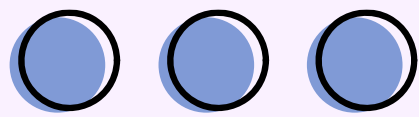
4) Manage Exceptions

5) try-with-resources

6) Custom Exception

7) Supplementary Contents



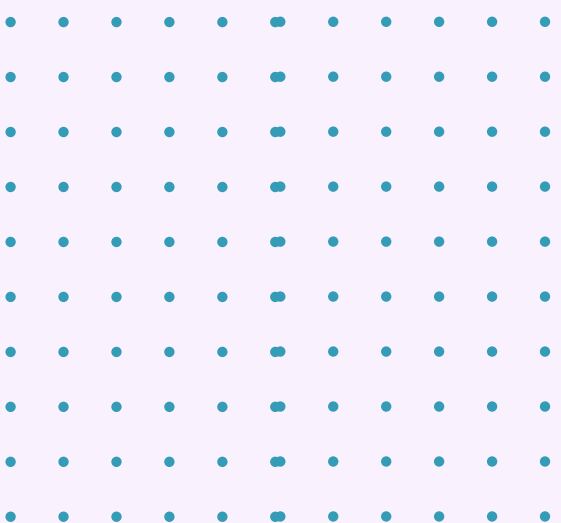
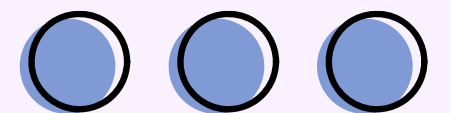


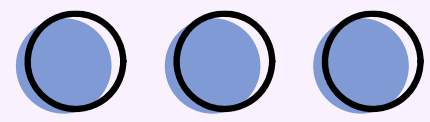
Bugs | Errors | Exceptions

1) **Bugs** come in many forms. Some bugs do not disrupt program execution, but eventually return incorrect results. For example, if there is a mistake in the tax calculation formula for a shop's products, an incorrect amount will be computed. This type of defect is referred to as a logical bug. There is also a class of messy code that leads to memory leaks and resource wastage.

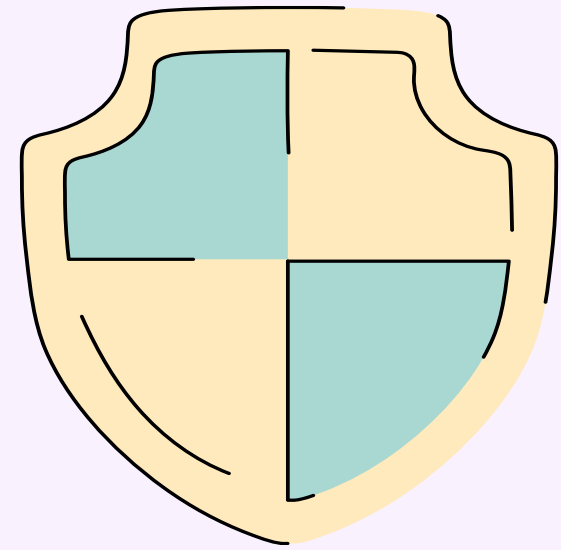
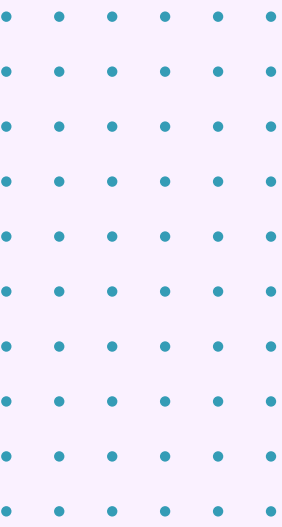
2) **Errors** are a broader category and can be subdivided into various types. For instance, User Errors are primarily caused by the user's improper use of the program. An example would be if the data entered by a user into form fields has a format different from what the system expects.

3) **Exceptions** are a mechanism for handling certain runtime problems. They tend to be less intuitive than other types of events when programming. Imagine a piece of code that attempts to connect to an external source to fetch data. If that external source (file or database) is deleted or corrupted and the connection cannot be established properly, an Exception occurs.





Some Examples about Exceptions



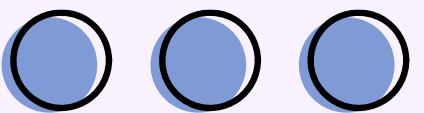
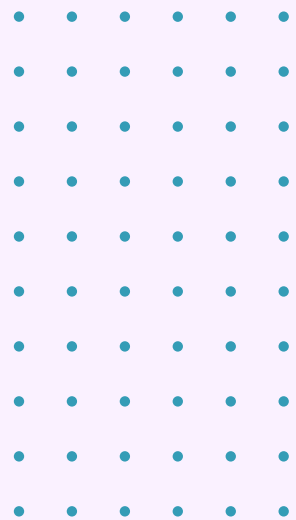
Java provides an effective solution for handling common coding and runtime issues through Exceptions. An exception occurs when an unexpected problem or error arises in a program.

Dividing a number by zero : `ArithmeticException`

Accessing an element in an array with an invalid index: `ArrayIndexOutOfBoundsException`

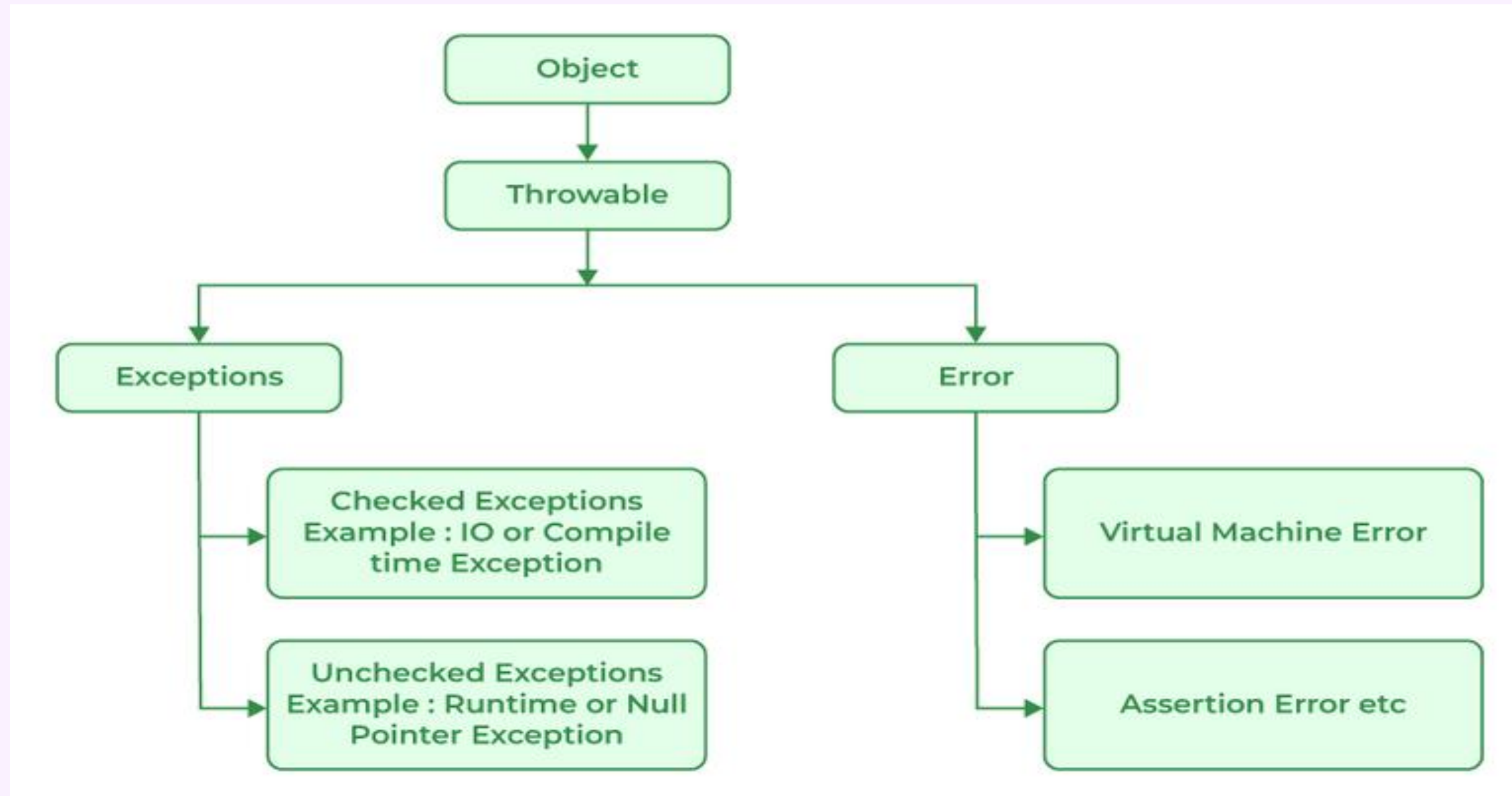
Attempting to open a file that does not exist :
`IOException`

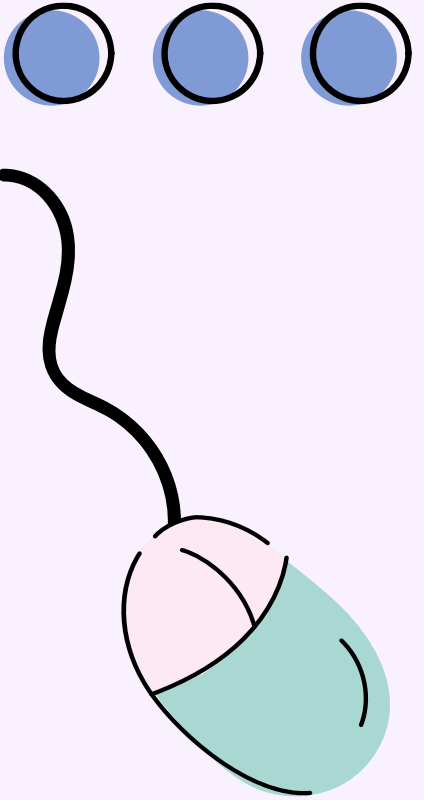
Database or network connection errors.



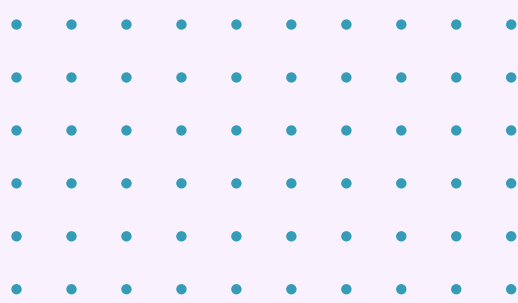
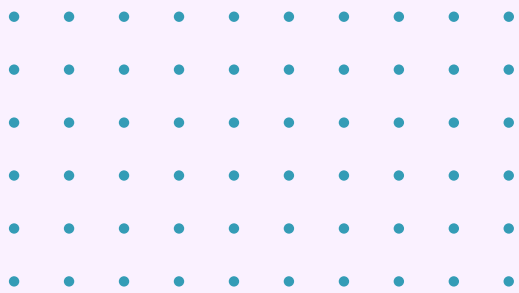


Exception's Hierarchy in Java

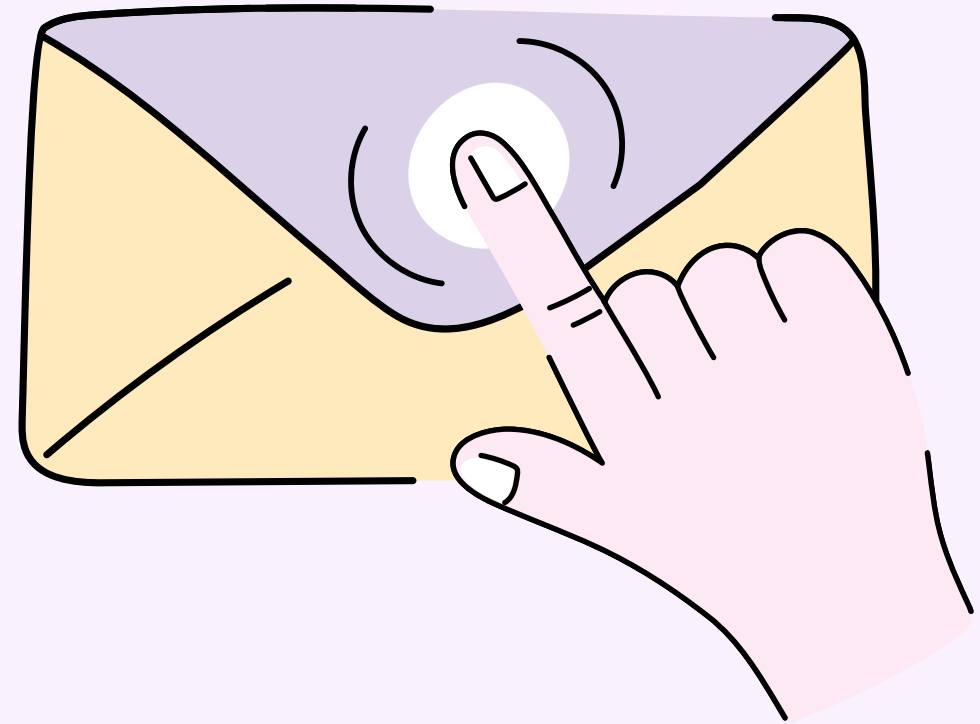


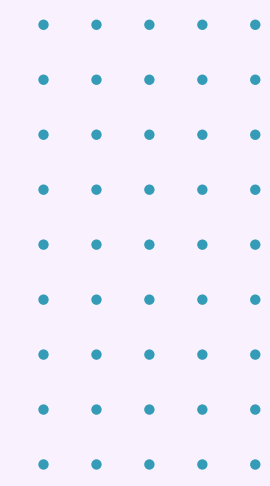
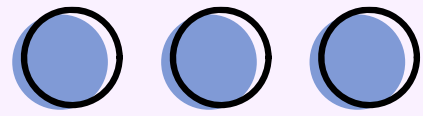


Error : This category contains errors that are usually outside the programmer's control and should not normally be managed, such as memory-related errors(such as **OutOfMemoryError** and **StackOverflowError**). Errors are thrown by JVM or libraries to show the errors associated with the execution time environment (JRE). In fact, the errors represent serious and usually non-recoverable conditions such as libraries, infinite return, or memory leak.

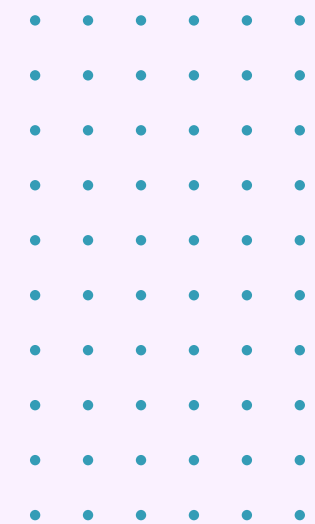
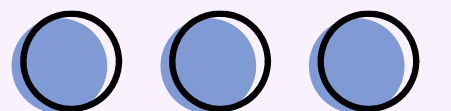


Exception : This category refers to exceptions, which are somehow predictable and managed by the programmer .
Checked Exceptions: Contains exceptions that require management (from class Exception) .
Unchecked Exceptions / Runtime Exception : Includes exceptions that occur at runtime and do not necessarily need to manage (from class RuntimeException) .



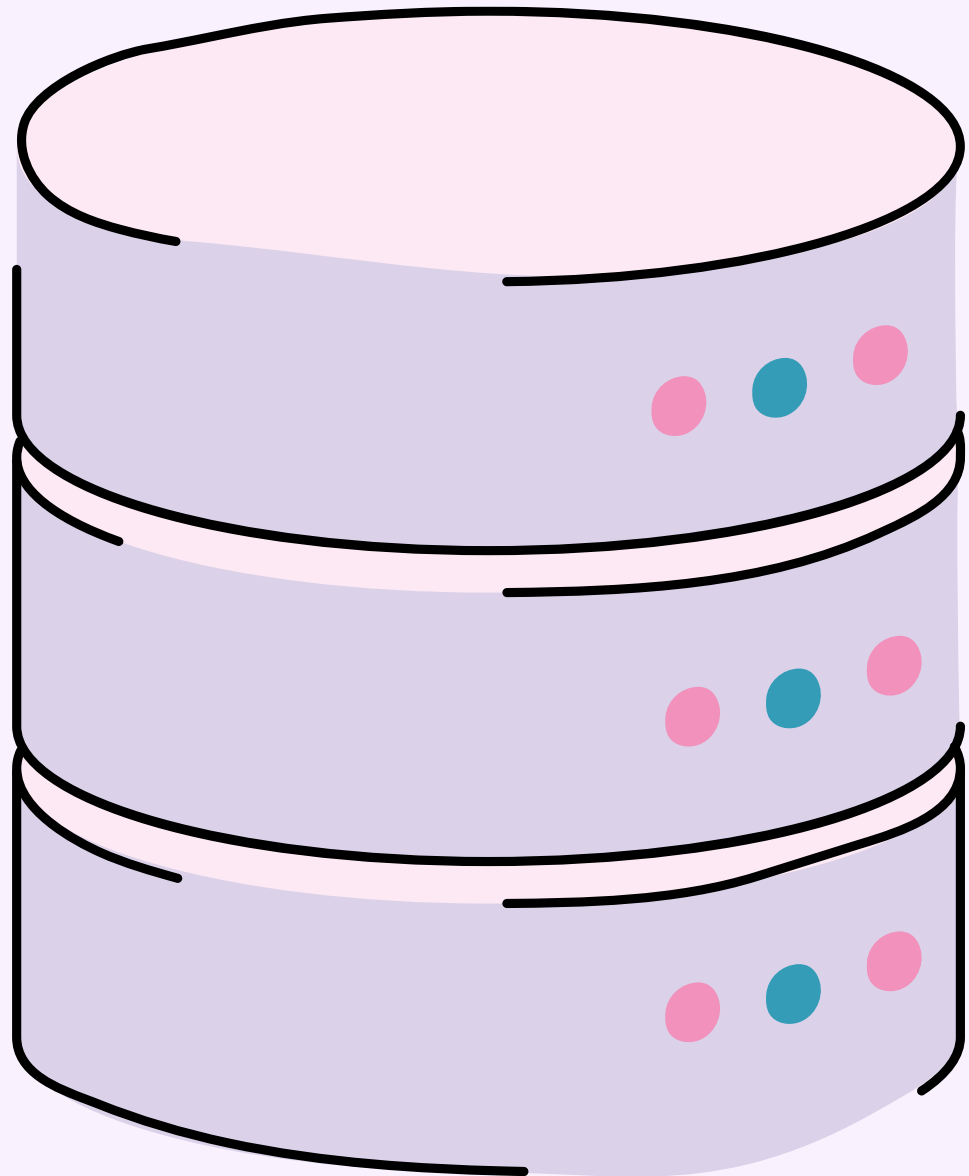


Checked Exceptions & Unchecked Exceptions



Checked Exceptions

These exceptions require the programmer to act to **manage them otherwise not compile**. For example, when we work with files or databases, we may encounter problems such as having no access to a file or cut off with a database. The oracle documentation advises us to use checked exceptions when we expect that the methodology can reasonably satisfy the situation . Examples of exceptions include : **IOException** and **ServletException** .

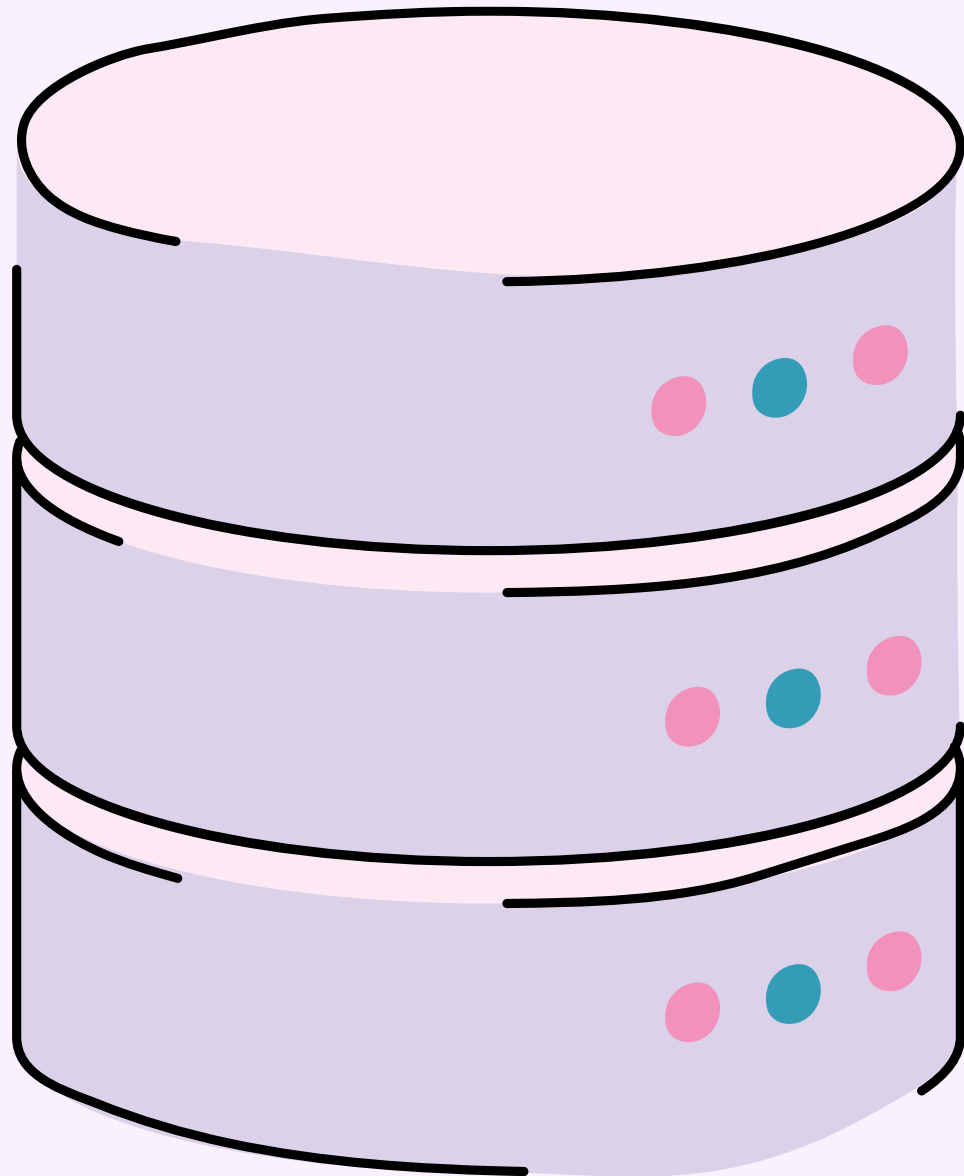


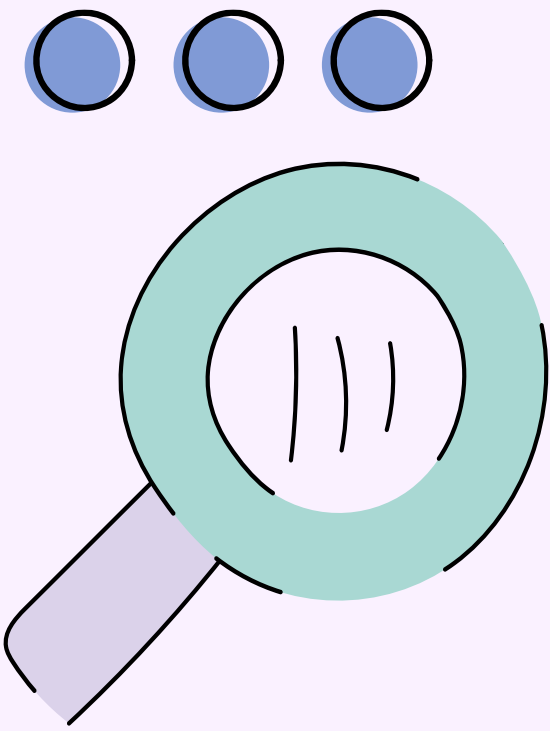
Unchecked Exceptions

Unchecked exceptions are exceptions that the java compiler does not require handling or declaring them. These errors usually occur for reasons such as wrong in coding, such as division by zero or access to an invalid index in the array. These types of errors are managed optional, and usually do not need to be declared with throws.

Some examples of Unchecked exceptions include :

`NullPointerException`,
`IllegalArgumentException`, and
`SecurityException`.





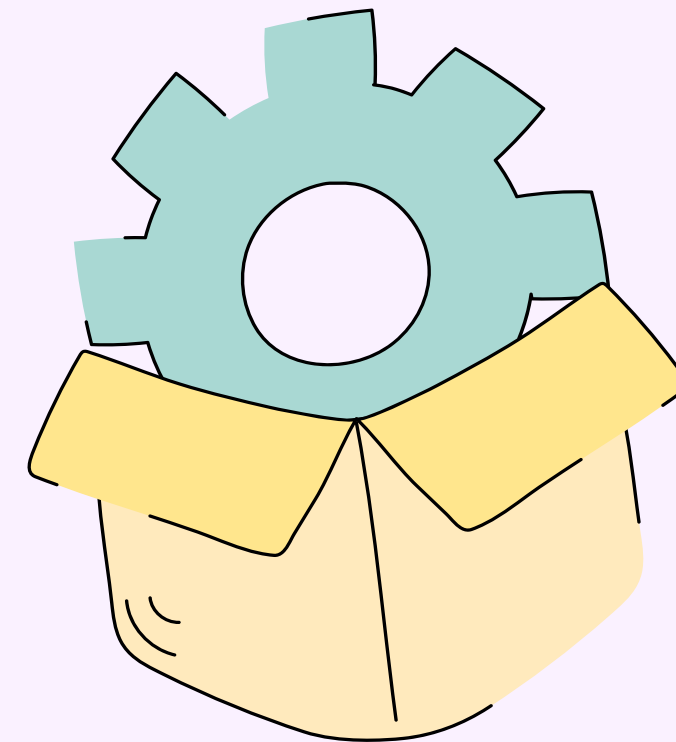
Try

Catch

Finally

Throw

Throws

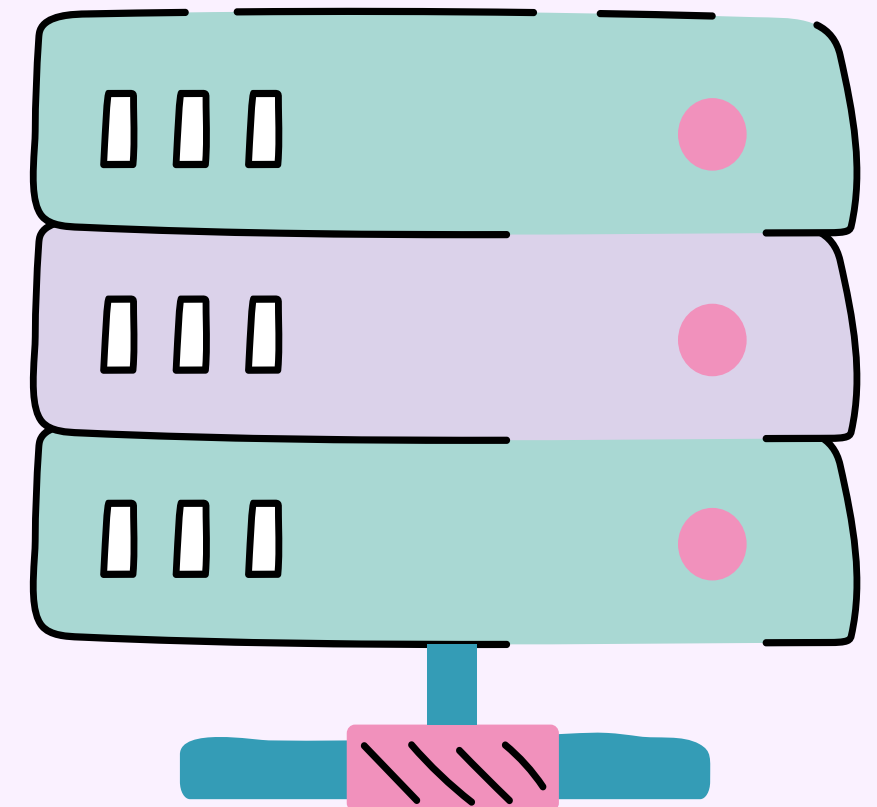


Try-Catch Blocks

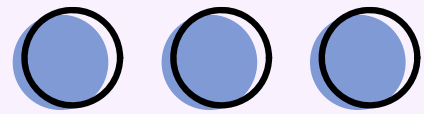
Try: part of the code that is likely to be error - prone.

Catch: part of the code that is executed when an exception occurs and is mainly referred to as error.

```
1  import java.lang.Exception;
2  import java.util.Scanner;
3
4  public class Calculator { 1usage
5      public static int operate(int number) throws Exception { 1usage
6          if(number > 10){
7              throw new Exception("The number entered is greater than the specified range .");
8          }
9          return number/= 2 ;
10     }
11 }
12 class Calc_Test {
13     public static void main(String[] args){
14         Scanner scanner = new Scanner(System.in) ;
15         try {
16             int number = scanner.nextInt();
17             int answer = Calculator.operate(number);
18             System.out.println(answer);
19         }
20         catch(Exception exception){
21             System.out.println(exception.getMessage());
22         }
23     }
24 }
```



Throw and Throws



Throw: When we want to notify the availability of a fault or problem in a particular operation in services or methods, we must notify it of the term or service using an exception, so to speak the so-called exceptions.

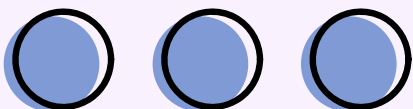
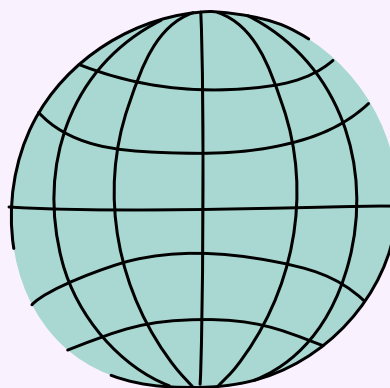
Throws: In the Java programming language, when a method may create and launch an exception, we need to inform consumers of the method that this method may throw such errors. To do so, we use the keyword throws.

Why we use throws?

If a method may launch checked exceptions, we must declare it using throws so that other parts of the program know that they must manage this error. This helps prevent problems during execution.

Syntax using throws

The key word throws is at the end of the signature. After that, we have to write the unique type that may be launched by the method.



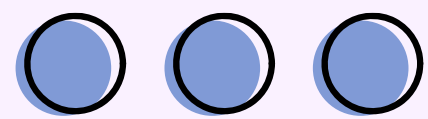


Throw and Throws

readFile
method may
create a
IOException
error.

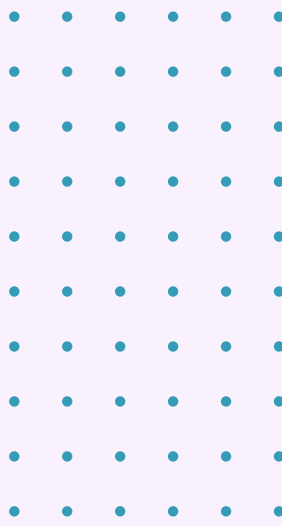
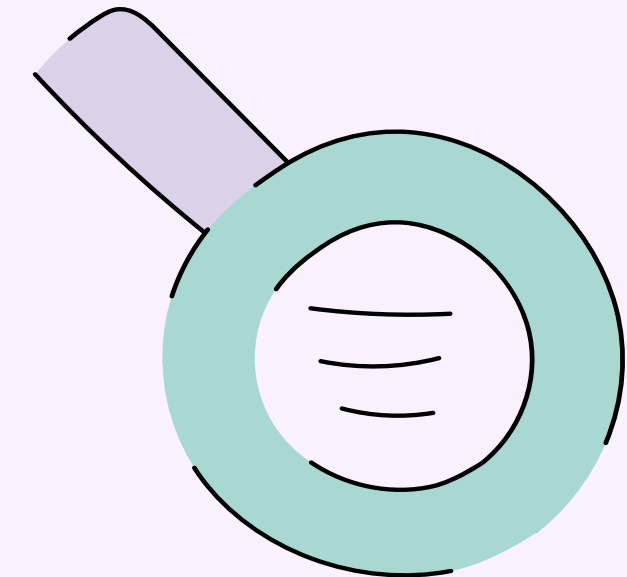
```
1  import java.io.BufferedReader;
2  import java.io.FileNotFoundException;
3  import java.io.FileReader;
4  import java.io.IOException;
5
6  public class FileException { no usages
7      public void readFile() throws IOException { no usages
8          FileReader fileReader = new FileReader( fileName: "file.txt" ) ;
9          BufferedReader bufferedReader = new BufferedReader(fileReader) ;
10         String line = bufferedReader.readLine() ;
11         System.out.println(line);
12     }
13 }
```





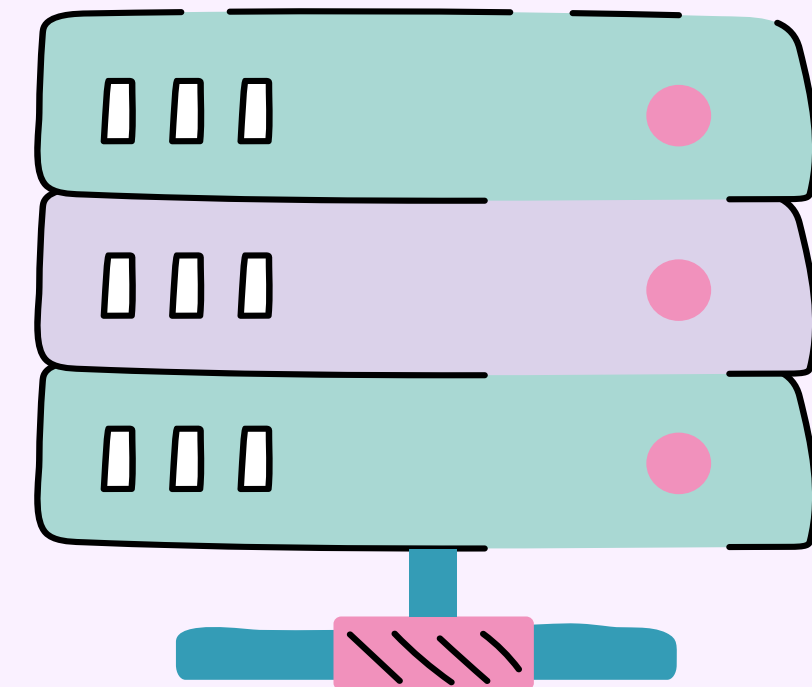
Finally

Sometimes we need to end up with a specific code , like when we read the contents of a file . In this case , we must close the file to open the file .



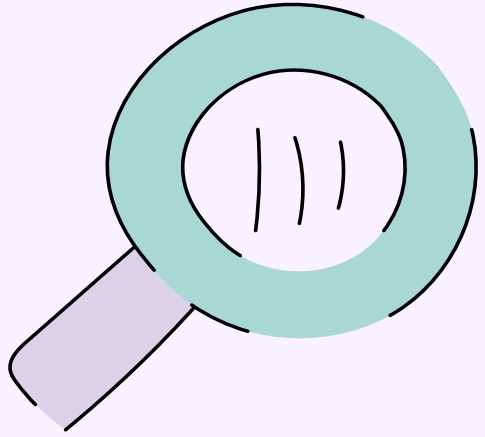
```
1 > public class FinallyException {  
2 >     public static void main(String[] args) {  
3         try {  
4             System.out.println("Opening file...");  
5             throw new Throwable("Error In Reading File");  
6         } catch (Throwable throwable) {  
7             System.out.println("Caught an exception: " + throwable.getMessage());  
8         } finally {  
9             System.out.println("Closing file...");  
10        }  
11    }  
12 }
```

In the example above, for simplicity of concept transfer, instead of directly using the file classes we will learn in the future, an error is launched within the try block, which simulates the error in reading from the file. Now, regardless of whether the error occurs or not, we observe that the expression is performed within the finally block.





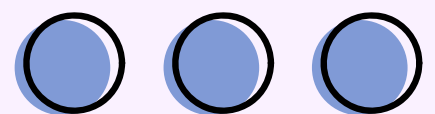
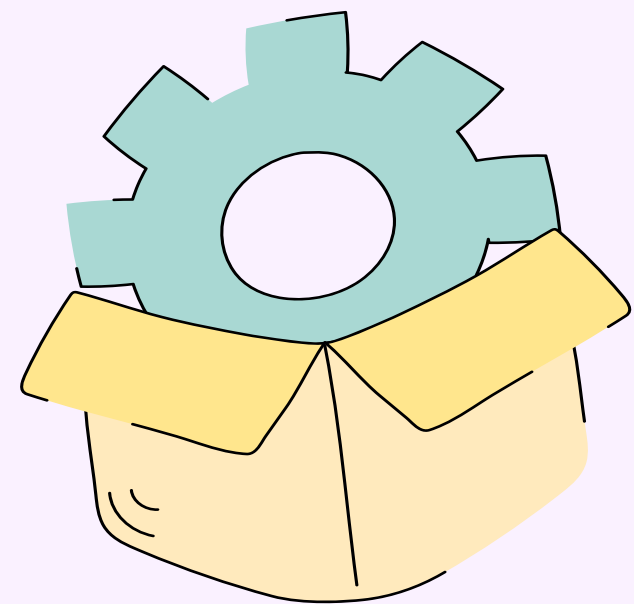
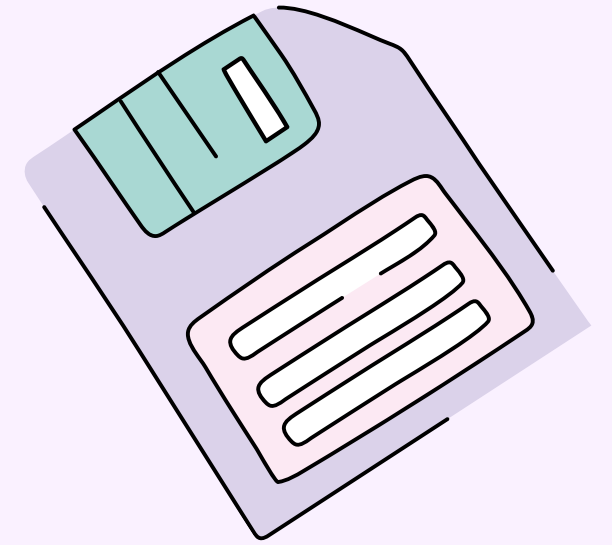
Try-With-Resources



In Java, When we use a resource(such as a file or connection to a database), we need to close it after finishing the work with that resource. This is usually done within the finally block. But with other resources, there is no need to write an additional code to close the resources. Resources that are defined within the block diagram are automatically closed after the end of use.

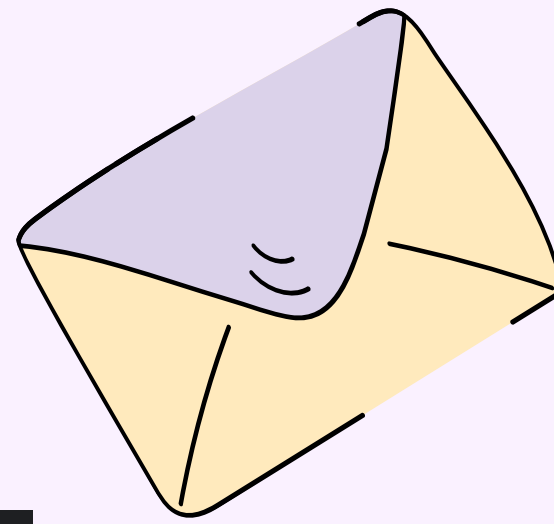
```
try (ResourceType resource = new ResourceType()) {  
    // use the resource  
} catch (ExceptionType e) {  
    // handle exceptions  
} finally {  
    // optional code to run after try or catch block  
}
```

The resources defined in the bracket after the word **try** are automatically closed after the use is completed. The word **finally** still exists, but there is no need to close the resources.

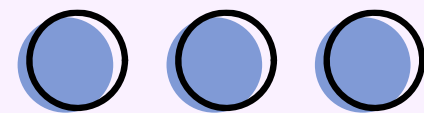
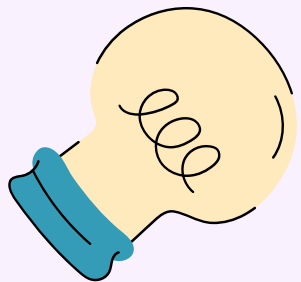




Try-With-Resources



```
1  import java.io.BufferedReader;
2  import java.io.FileNotFoundException;
3  import java.io.FileReader;
4  import java.io.IOException;
5
6  ▶ public class MultipleResourcesExample {
7  ▶     public static void main(String[] args) throws FileNotFoundException {
8         try(FileReader fileReader = new FileReader( fileName: "File.txt") ;
9         BufferedReader bufferedReader = new BufferedReader(fileReader)){
10             //codes to use resources ;
11         }
12         catch(IOException ioException){
13             ioException.printStackTrace();
14         }
15     }
16 }
```



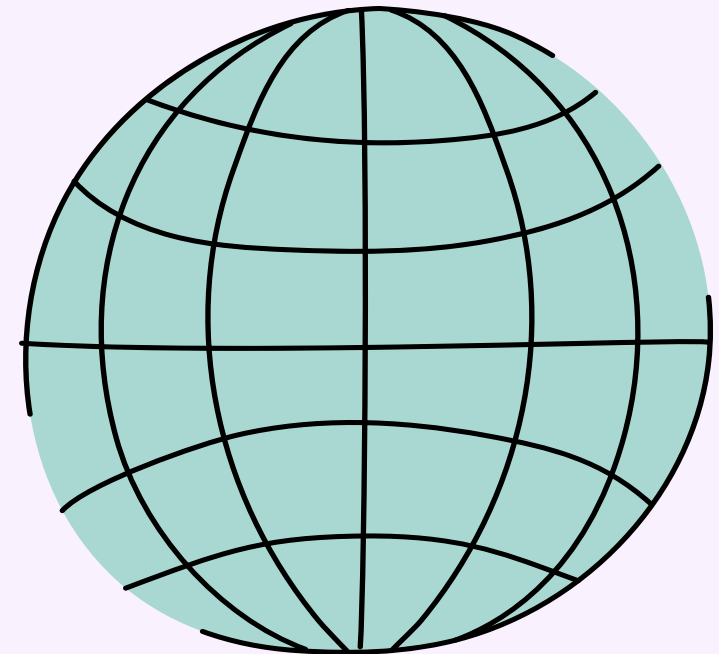
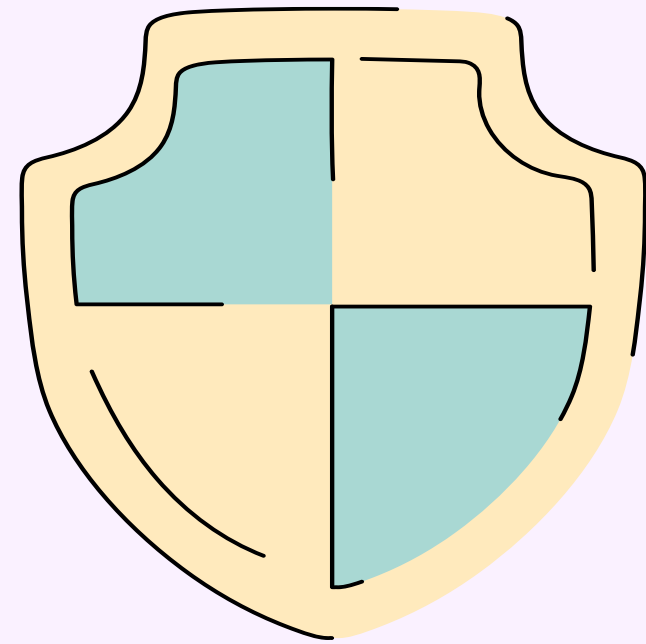


Benefits of using try-with-resources

1) Additional code reduction:
No need to write code for closing resources in the finally block.

2) More Security:
Resources are automatically closed, even if there is an error while using them.

3) More Readability: The use of options- with-resources makes the code easy and applicable.





Tips and Limitations try-with-resources

1) **Implementation of AutoCloseable interface**: Classes we want to use in try-with-resources must have implemented **AutoCloseable interface**. This interface replaces the older Closeable interface and has a method called close (), which is called the call after the work is over.

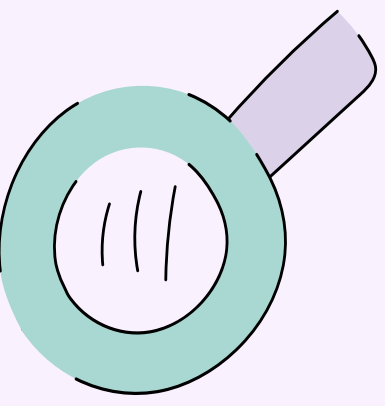
2) **Error in closing resources**: If there is an error when closing resources, this error is added to the original error and both are displayed .

3) **Automatic closing of resources** : The resources placed in the try block will automatically be closed after the application is complete .

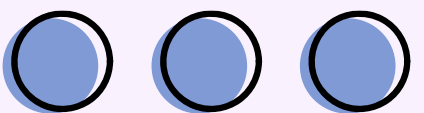
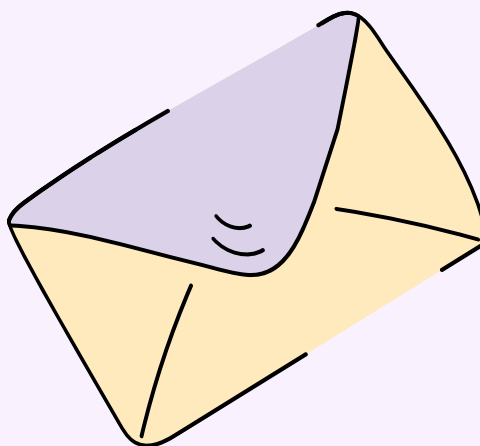




Custom Exception



There are errors that we create by inheriting from the base class "**Exception**" (or its subclasses), and we can customize them by overriding the base class's methods and attributes. This type of error enables the programmer to define their own specific exceptions tailored to the program's needs and to use them for better **error handling**.





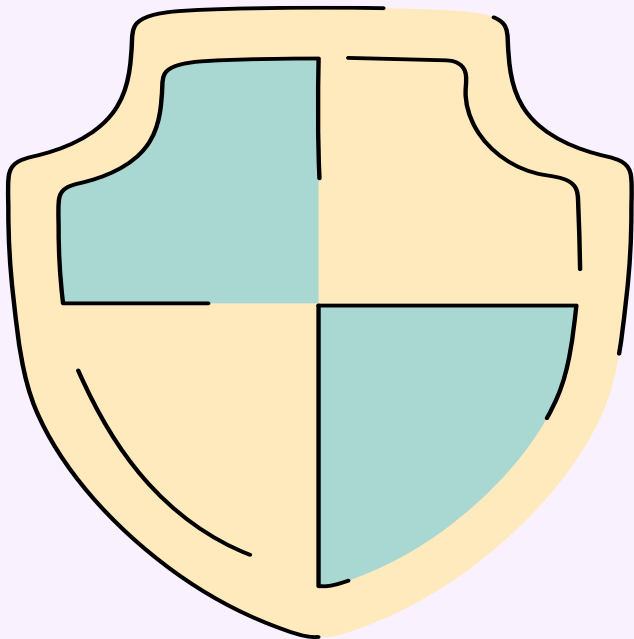
1

```
import java.lang.Exception;

public class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}
```

2

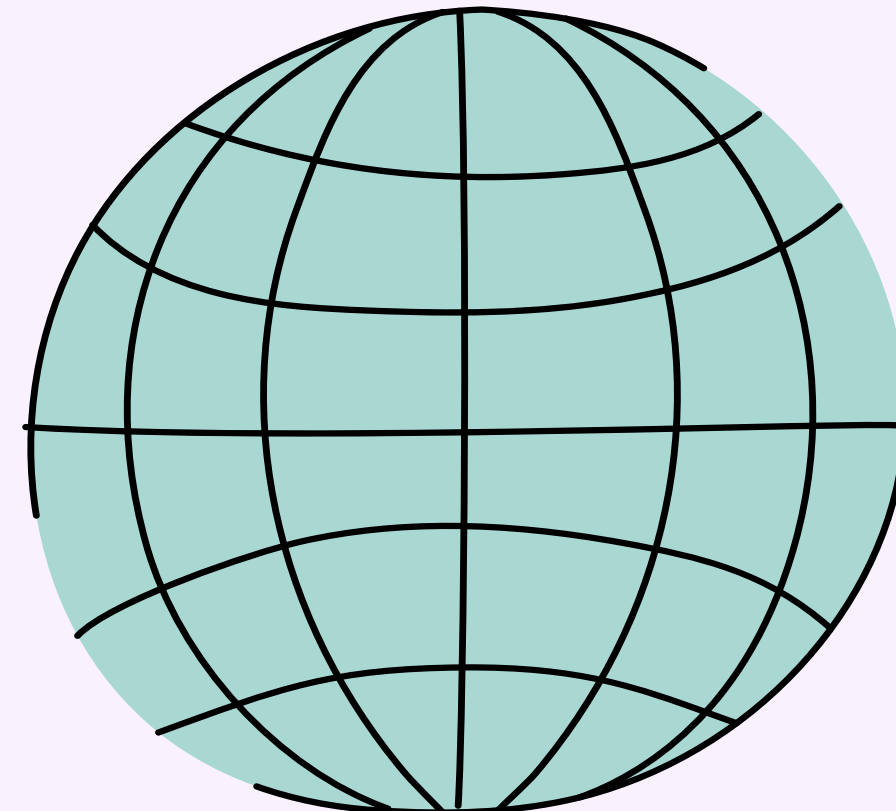
```
public class AgeValidator {
    public static void validateAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or older");
        } else {
            System.out.println("Age is valid.");
        }
    }
}
```



```
class Age_Validate {
    public static void main(String[] args) throws InvalidAgeException {
        AgeValidator.validateAge(16);
    }
}
```

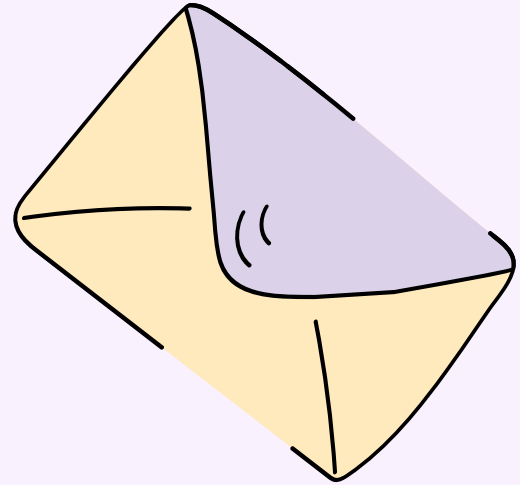
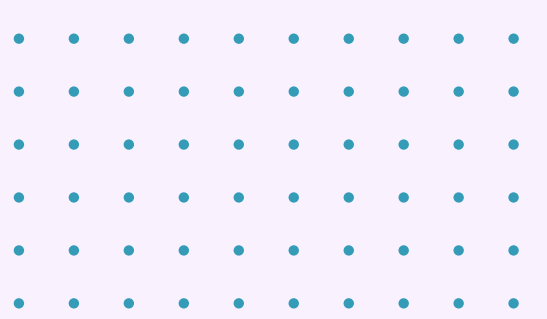
Age_Validate x

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetB
Exception in thread "main" InvalidAgeException: Age must be 18 or older
at AgeValidator.validateAge(AgeValidator.java:4)
at Age_Validate.main(AgeValidator.java:14)





Why we use custom Exception?



1) More Precise Error Handling:

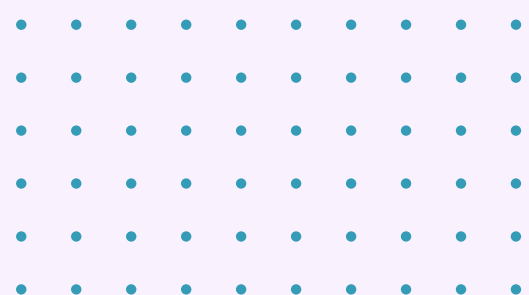
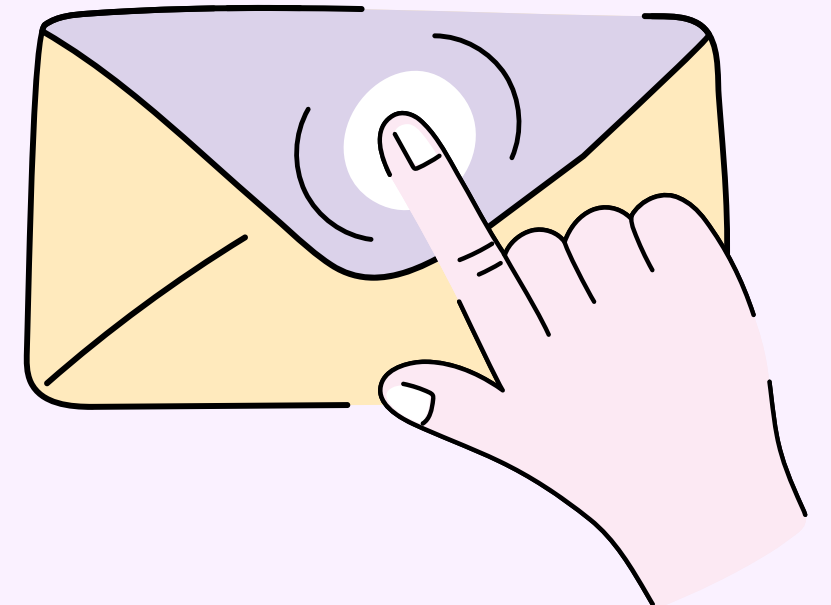
In complex applications, we may need to identify a particular type of error separately. Using customized exceptions allows errors to be managed more precisely.

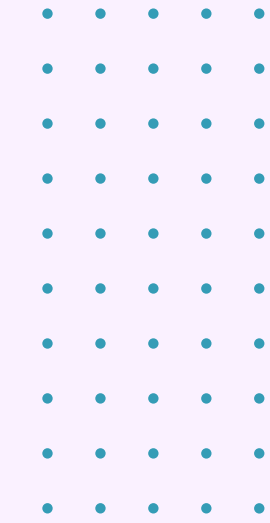
2) Cleaner Code:

When we create a specific exception for particular conditions, the code becomes more readable and understandable. This is especially useful in large projects.

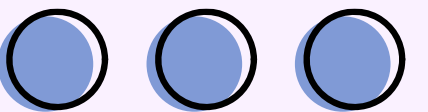
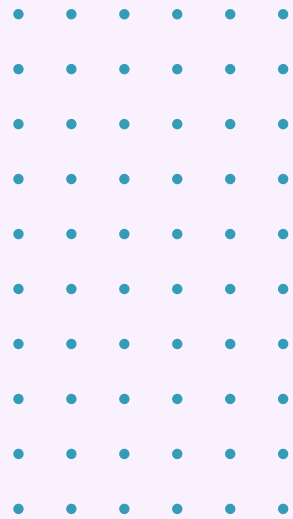
3) Greater Flexibility:

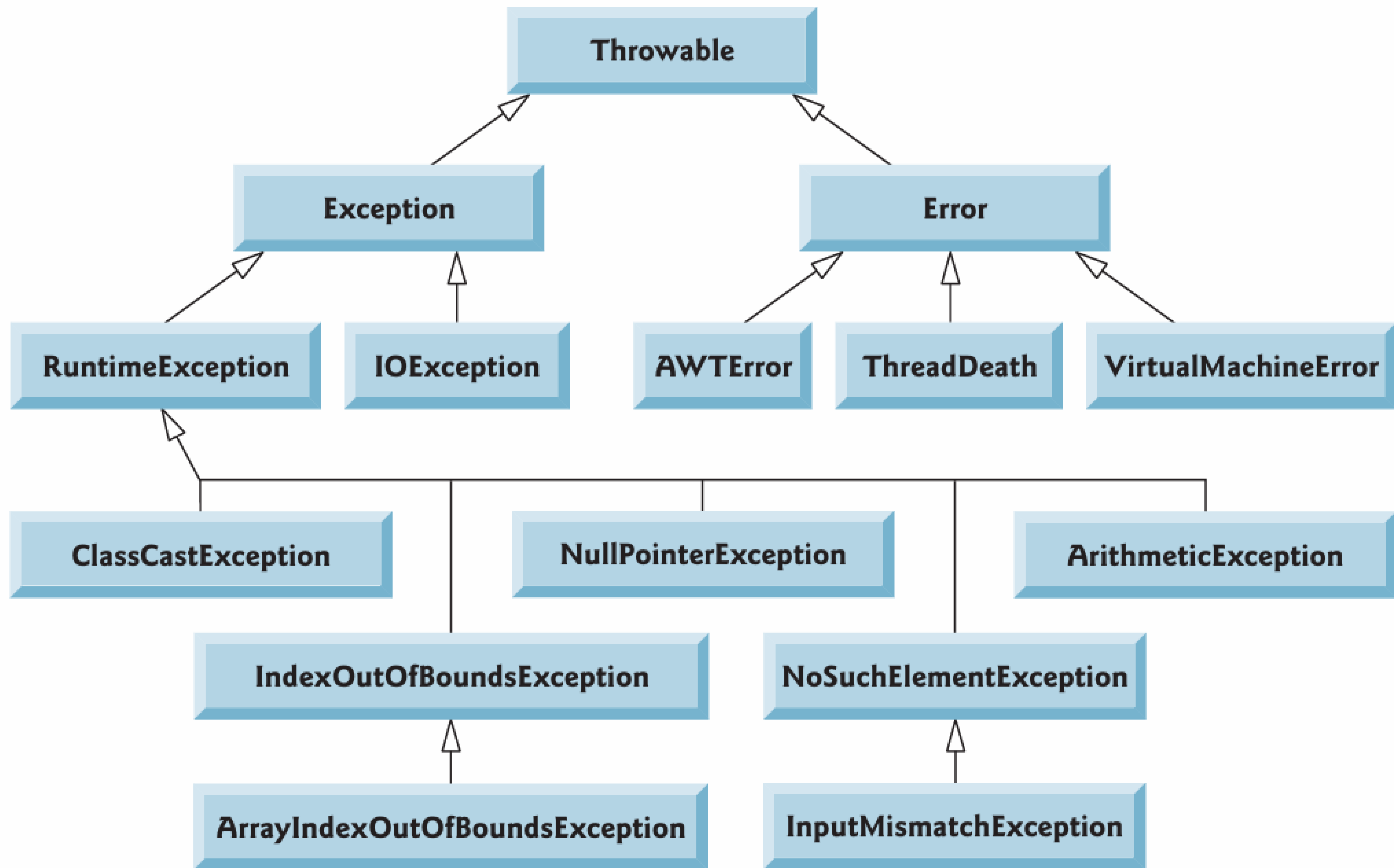
By defining custom exceptions, we can perform specific actions when an error occurs that might not be possible with Java's standard exceptions.





Exception Class Details







RuntimeExceptions are Unchecked Exceptions:
All exception types that are direct or indirect subclasses of `RuntimeException` (package `java.lang`) are unchecked exceptions. These are typically caused by defects in your program's code. Examples of unchecked exceptions include:

`ArrayIndexOutOfBoundsException` : You can avoid these by ensuring that your array indices are always greater than or equal to 0 and less than the array's length.

`ArithmeticException` : You can avoid the Arithmetic Exception that occurs when you divide by zero by checking the denominator to determine whether it's 0 before performing the calculation.

```
1 public class ArrayExample {
2     public static void main(String[] args){
3         int[] arr = {40 , 32 , 50 , 90 , 12} ;
4         try {
5             System.out.println("Element Of Fifth Index is : " + arr[5]);
6         }
7         catch (ArrayIndexOutOfBoundsException exception){
8             System.out.println(exception.getMessage());
9         }
10    }
11 }
```

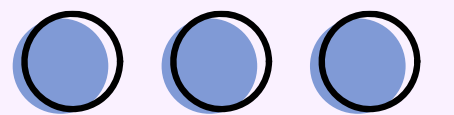
ArrayExample x

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBr
Index 5 out of bounds for length 5

```
1 public class ArithmeticExample {
2     public static void main(String[] args){
3         int numerator = 10 ;
4         int denominator = 0 ;
5         try {
6             int answer = numerator / denominator ;
7             System.out.println(answer);
8         }
9         catch(ArithmeticException exception){
10            System.out.println(exception.getMessage());
11        }
12    }
13 }
```

ArithmeticExample x

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Pro
/ by zero



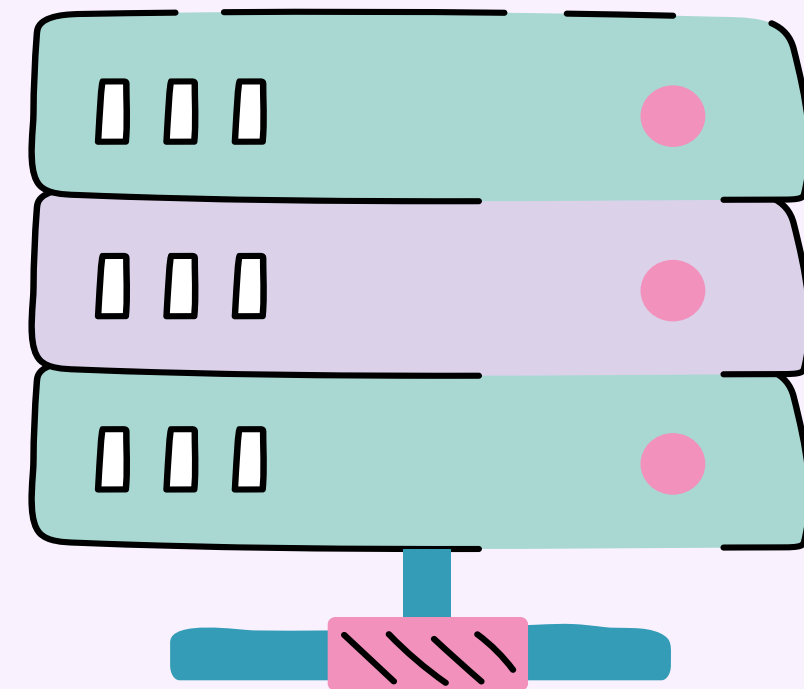
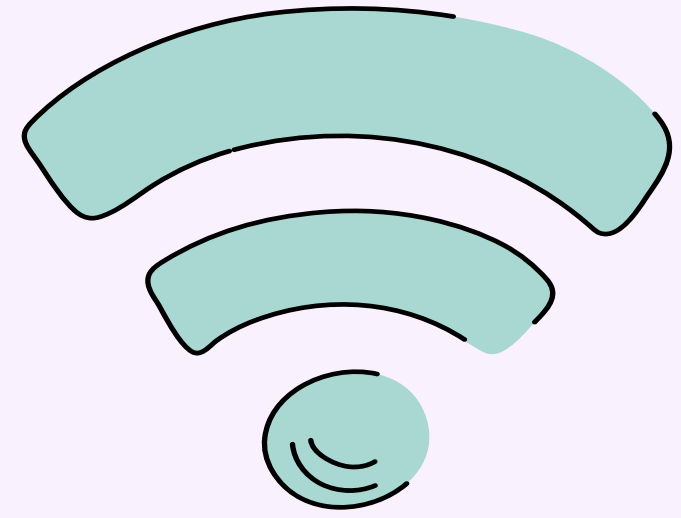


java.lang.Exception

1) Exception class in java is directly inherited from java.lang.Throwable.

For this reason , all methods of managing errors defined in Throwable are also available in Exception.

2)The Exception class does not add any particular method , but only constructors to create an exception.





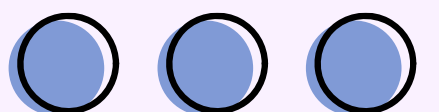
getCause()

```
1 ▶ public class CauseDemo {
2 ▶     public static void main(String[] args) {
3         try {
4             try {
5                 throw new IllegalArgumentException("Invalid Input");
6             } catch (IllegalArgumentException exception) {
7                 throw new RuntimeException("Higher Level Error", exception);
8             }
9         } catch (RuntimeException exception) {
10             System.out.println("Reason For Exception : " + exception.getCause().getMessage());
11         }
12     }
13 }
```

CauseDemo x

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Comm
Reason For Exception : Invalid Input

“A method wich return cause of exception”





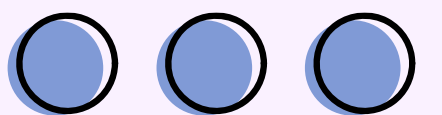
getMessage()

```
1 ▶ public class MessageDemo {
2 ▶     public static void main(String[] args) {
3         try {
4             throw new NullPointerException("Null Value in Wrong Place");
5         } catch (NullPointerException e) {
6             System.out.println("Error Message : " + e.getMessage());
7         }
8     }
9 }
```

MessageDemo x

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBra
Error Message : Null Value in Wrong Place

“Optional text message submitted when making an exception”





getStackTrace()

```
1 ▶ public class StackTraceExample {
2 ▶     public static void main(String[] args) {
3         try {
4             throw new RuntimeException("Wrong Test");
5         } catch (RuntimeException runtimeException) {
6             StackTraceElement[] trace = runtimeException.getStackTrace();
7
8             System.out.println("StackTrace Details : ");
9             for (StackTraceElement element : trace) {
10                 System.out.printf("- %s.%s (%s:%d)%n", element.getClassName(), element.getMethodName(), element.getFileName(), element.getLineNumber());
11             }
12         }
13     }
14 }
```

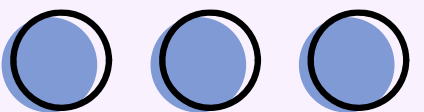
StackTraceExample x

⌂ 🔍 ⋮

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1.3\lib\idea_rt.jar=62769" -Dfile.encoding=UTF-8

StackTrace Details :
- StackTraceExample.main (StackTraceExample.java:4)

“Turns the array of stack lines”





setStackTrace(StackTraceElement[] stackTrace)

```
1  > public class StackTraceDemo {
2  >     public static void main(String[] args) {
3      try {
4          throw new Exception();
5      } catch (Exception exception) {
6          StackTraceElement custom = new StackTraceElement(
7              declaringClass: "CustomClass", methodName: "customMethod", fileName: "CustomFile.java", lineNumber: 42);
8          exception.setStackTrace(new StackTraceElement[]{custom});
9          exception.printStackTrace();
10     }
11 }
```

StackTraceDemo x

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1.3\T
Exception
at CustomClass.customMethod(CustomFile.java:42)

“Array sets the stack error elements”





printStackTrace()

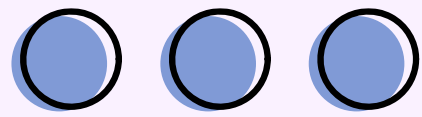
```
1 ▶ public class PrintStackDemo {
2 ▶     public static void main(String[] args) {
3         try {
4             int[] arr = new int[1];
5             System.out.println(arr[5]);
6         } catch (ArrayIndexOutOfBoundsException e) {
7             System.err.println("Error off the index outside the scope : ");
8             e.printStackTrace();
9         }
10    }
11 }
```

PrintStackDemo x

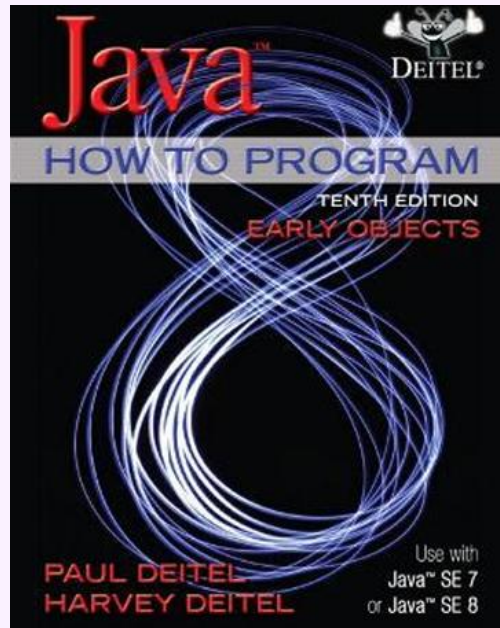
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ ID
Error off the index outside the scope :
java.lang.ArrayIndexOutOfBoundsException Create breakpoint : Index 5 out of bounds for length 1
at PrintStackDemo.main(PrintStackDemo.java:5)

“The stack is printed in System.err”





Resources



Java How to Program (Deitel)

Chapter 11 Exception Handling

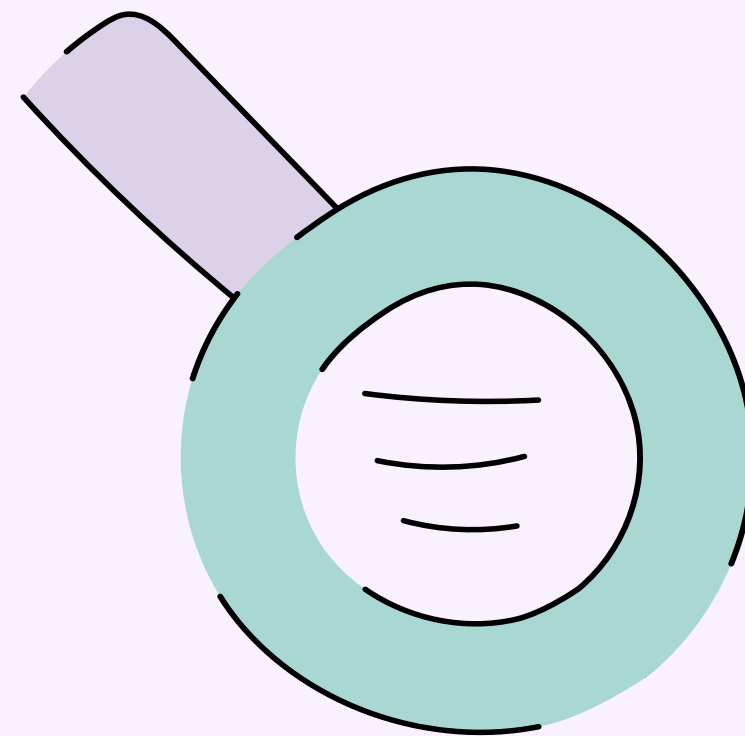
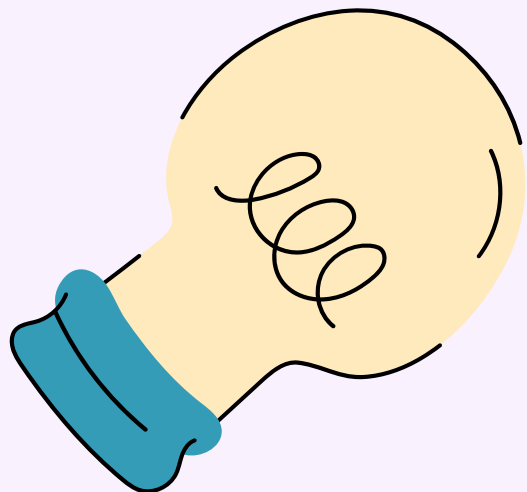
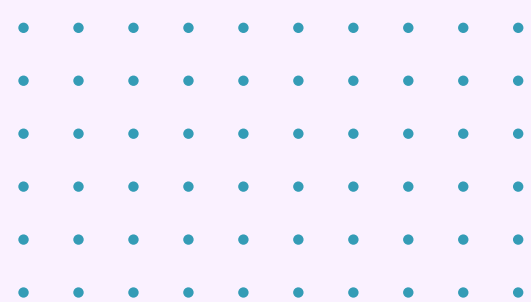


Project-Oriented Course in Advanced Java Programming



Exception Handling





Thanks!

